

AI-DRIVEN PROCESS SCHEDULING SYSTEM

OPERATING SYSTEMS (CS23501)

PROJECT DOCUMENTATION

Team Members

Name	Register Number
Ajay Kumar S	2024503701
Sudalai Kumar M	2024503703
Srinivasan S	2024503707
Satlas Rohit B	2024503305

Computer Technology

Batch-A

08 October 2026

ABSTRACT

The AI-Driven Process Scheduling System is an experimental web-based CPU scheduling platform that combines conventional scheduling algorithms with machine learning and reinforcement learning. The application provides a browser interface through which users can enter simulated process information and compare traditional scheduling behaviors with an AI-based scheduling workflow. The traditional side implements First-Come First-Serve (FCFS), Shortest Job First (SJF), and Round Robin in client-side JavaScript. The AI side sends process features to a Flask backend, where a Random Forest Regressor is used to predict burst time and a custom ProcessSchedulingEnv is used with a Proximal Policy Optimization (PPO) agent to choose between continuing the current process and switching processes.

The machine-learning component uses process-related features from the project dataset and applies feature scaling before Random Forest prediction. The reinforcement-learning state contains predicted burst time, waiting time, queue length, and progress. The action space contains two actions: continue and switch. The reward design provides a positive reward for process completion and penalties for context switching and excessive waiting. The system returns scheduling information and performance metrics to the web interface.

The project is intended as a simulation and educational platform rather than a production operating-system scheduler. Its current architecture does not directly modify an operating-system kernel or schedule real OS processes. The implementation also uses local CSV and serialized model files instead of a persistent relational database. Further validation with independent real-world traces would be required before making claims about generalization or production scheduling performance.

TABLE OF CONTENTS

The following chapter structure is used in this revised document. In Microsoft Word, the table of contents can be refreshed using References → Update Table if Word displays the TOC field.

1. Introduction
2. Existing System
3. Proposed System
4. Requirements Analysis
5. Technology Stack
6. System Architecture
7. System Design
8. Functional Modules
9. AI/ML and Reinforcement Learning Implementation
10. Data Storage Design
11. API Design
12. User Interface
13. Implementation Details
14. Security
15. Testing
16. Deployment
17. Limitations
18. Future Enhancements
19. Conclusion
20. References

1. INTRODUCTION

1.1 Background

CPU scheduling is a fundamental operating-system activity that determines which ready process receives CPU execution time. Classical algorithms such as FCFS, SJF, and Round Robin use explicit scheduling rules. This project studies whether machine learning and reinforcement learning can be combined with a scheduling simulator to make adaptive scheduling decisions.

1.2 Problem Statement

Traditional scheduling policies are based on fixed rules and may behave differently as workload characteristics change. In particular, practical scheduling decisions may involve uncertain or predicted process behaviour. The project therefore explores a simulation in which process-related features are used to estimate burst time and an RL agent learns a scheduling action.

1.3 Motivation

The motivation is to connect operating-system scheduling concepts with modern AI techniques. The project provides a visual environment in which traditional scheduling results can be compared with an AI-based scheduling workflow without requiring changes to the host operating system kernel.

1.4 Objectives

- Implement FCFS, SJF, and Round Robin scheduling in the browser interface.
- Use a Random Forest Regressor to estimate process burst time from process-related features.
- Use a custom Gym-style environment and PPO to select continue/switch actions.
- Provide scheduling metrics through a web interface.
- Create an experimental platform for studying ML and RL in CPU scheduling.

1.5 Scope

The current scope is a simulated CPU scheduling environment. The system does not implement a kernel-level scheduler, does not directly control real operating-system processes, and does not provide a persistent user database. Its primary purpose is experimentation, visualization, and academic study.

1.6 Target Users

- Computer science students studying operating systems and AI.
- Researchers or project developers experimenting with AI-based scheduling simulations.
- Developers interested in combining supervised learning with reinforcement learning for scheduling.

1.7 Applications

- Educational CPU scheduling simulation.
- Experimental comparison of traditional and AI-based scheduling.
- Prototype research for intelligent scheduling policies.

2. EXISTING SYSTEM

2.1 Existing Approach

The conventional baseline consists of FCFS, SJF, and Round Robin. FCFS schedules processes in arrival order. SJF selects the process with the shortest burst time under the assumptions of the simulated workload. Round Robin uses a time quantum to provide time-sliced execution.

2.2 Problems and Limitations

Fixed scheduling rules do not learn from workload characteristics. SJF requires burst-time information, which is not generally known exactly in advance in a real system. Round Robin behaviour depends on the chosen time quantum. These characteristics motivate the project's experimental AI-based approach.

2.3 Important Qualification

The proposed application should not be described as replacing a real operating-system scheduler. It is a simulation platform. Therefore, statements about production CPU efficiency, real-time telemetry, or optimality should be treated as future research claims unless supported by separate real-system experiments.

3. PROPOSED SYSTEM

3.1 Overview

The proposed system has two execution paths. The Basic Scheduler path performs FCFS, SJF, and Round Robin calculations in the browser. The AI-Powered Scheduler path sends process features to Flask through the /schedule endpoint. The backend uses the machine-learning burst predictor and the PPO-based reinforcement-learning environment.

3.2 Key Features

- Browser-based process input for conventional scheduling.
- AI scheduler input containing arrival time and process-related feature values.
- Random Forest burst-time prediction.
- Custom ProcessSchedulingEnv for RL scheduling.
- PPO agent with continue/switch actions.
- Display of execution schedule and scheduling metrics.

3.3 AI/ML Terminology

The project uses machine learning and reinforcement learning. No Large Language Model (LLM) is required for the documented scheduling workflow. Therefore, the former section title 'AI/LLM Implementation' has been corrected to 'AI/ML and Reinforcement Learning Implementation'.

4. REQUIREMENTS ANALYSIS

4.1 Functional Requirements

1. The system shall allow users to enter basic process information for conventional scheduling.
2. The system shall allow users to select FCFS, SJF, or Round Robin in the Basic Scheduler interface.

3. The system shall allow users to enter AI scheduler process features.
4. The system shall send AI scheduling requests to the Flask /schedule endpoint.
5. The system shall predict burst time using the trained Random Forest model.
6. The system shall execute the PPO-based scheduling environment.
7. The system shall display scheduling results and available performance metrics.

4.2 Non-Functional Requirements

- Usability: the interface should allow process entry and result visualization without command-line interaction.
- Maintainability: the scheduler, ML model, RL environment, and UI are separated into project modules.
- Portability: the application is designed to run as a Python/Flask local web application.
- Scalability: the current project is a simulation and should not be interpreted as a demonstrated production-scale scheduler.

4.3 Hardware and Software Requirements

Category	Requirement
Processor	A modern x86-64 processor suitable for Python/ML execution
Memory	Sufficient RAM for Python, scikit-learn and Stable-Baselines3
Operating System	Windows, Linux, or another OS supporting the Python stack
Runtime	Python environment compatible with the project's dependencies
Browser	Modern web browser with JavaScript enabled

5. TECHNOLOGY STACK

Layer	Technology	Role
Frontend	HTML5	Application interface structure
Frontend	CSS3	Visual styling and layout
Frontend	JavaScript	Client-side scheduling logic and API calls
UI	Bootstrap 5	Responsive UI components/layout
Backend	Python + Flask	Web server and AI scheduling endpoint
ML	scikit-learn	Random Forest and feature scaling
RL	Stable-Baselines3	PPO implementation

RL Environment	Gym-compatible environment	Scheduling simulation
Data	Pandas / NumPy	Dataset and numerical processing
Model storage	Joblib	Serialization of ML predictor/scaler

5.1 Project Files

- `app.py` — Flask application and AI scheduling API.
- `templates/index.html` — web interface and client-side scheduling logic.
- `static/style.css` — application styling.
- `models/ml\_burst.py` — burst-time prediction model training/loading logic.
- `scheduler/rl\_scheduler.py` — custom RL environment and PPO scheduling logic.
- `data/process\_data.csv` — process data used by the ML component.
- Serialized model artifacts are stored under the models directory.

6. SYSTEM ARCHITECTURE

The actual implementation is a hybrid client-server design. The Basic Scheduler path is executed in browser-side JavaScript, whereas the AI scheduling path communicates with Flask through the /schedule endpoint. The backend then connects the burst-time prediction stage with the RL scheduling environment.

High-Level System Architecture

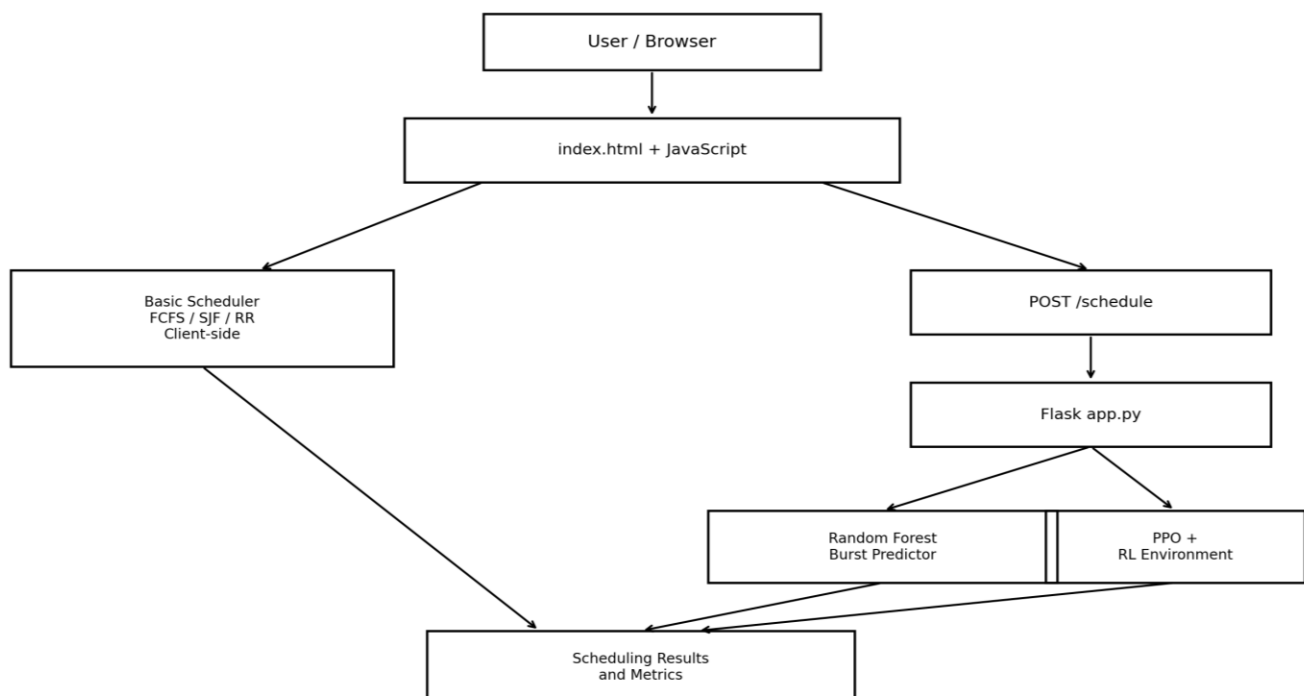


Figure 6.1 — Corrected High-Level System Architecture Diagram

6.1 Architecture Components

Component	Responsibility
Browser / index.html	Collects user input, runs conventional scheduler logic, sends AI requests, and displays results.
Basic Scheduler	Client-side FCFS, SJF, and Round Robin calculations.
Flask app.py	Receives AI scheduling requests and coordinates prediction/RL execution.
Random Forest Predictor	Predicts burst time from the AI feature set.
ProcessSchedulingEnv	Maintains simulated scheduling state and reward logic.
PPO Agent	Chooses continue or switch actions in the RL environment.
JSON Response	Carries AI scheduling results and metrics back to the browser.

7. SYSTEM DESIGN

7.1 Data Flow Diagram

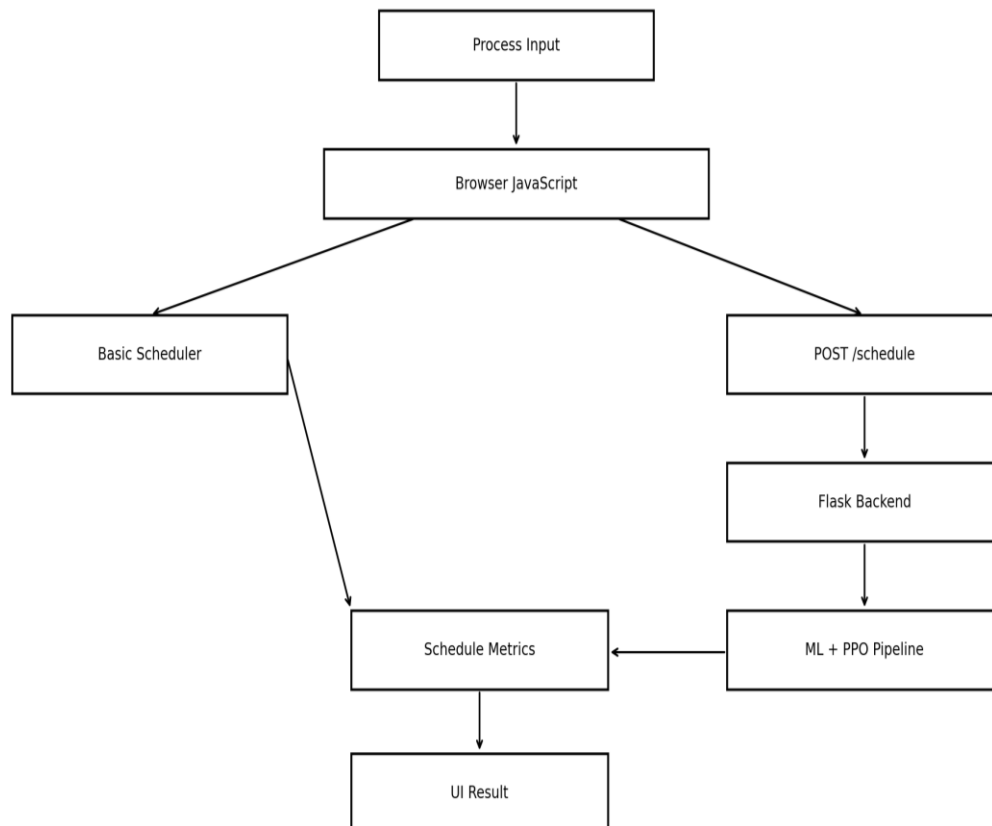


Figure 7.1 — Corrected Application Data Flow Diagram

7.2 Use Case Diagram

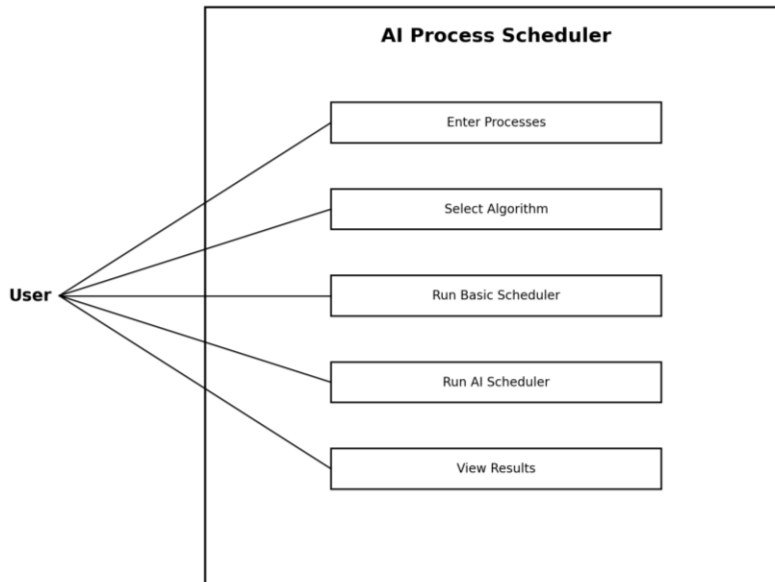


Figure 7.2 — Use Case Diagram

7.3 Sequence Diagram

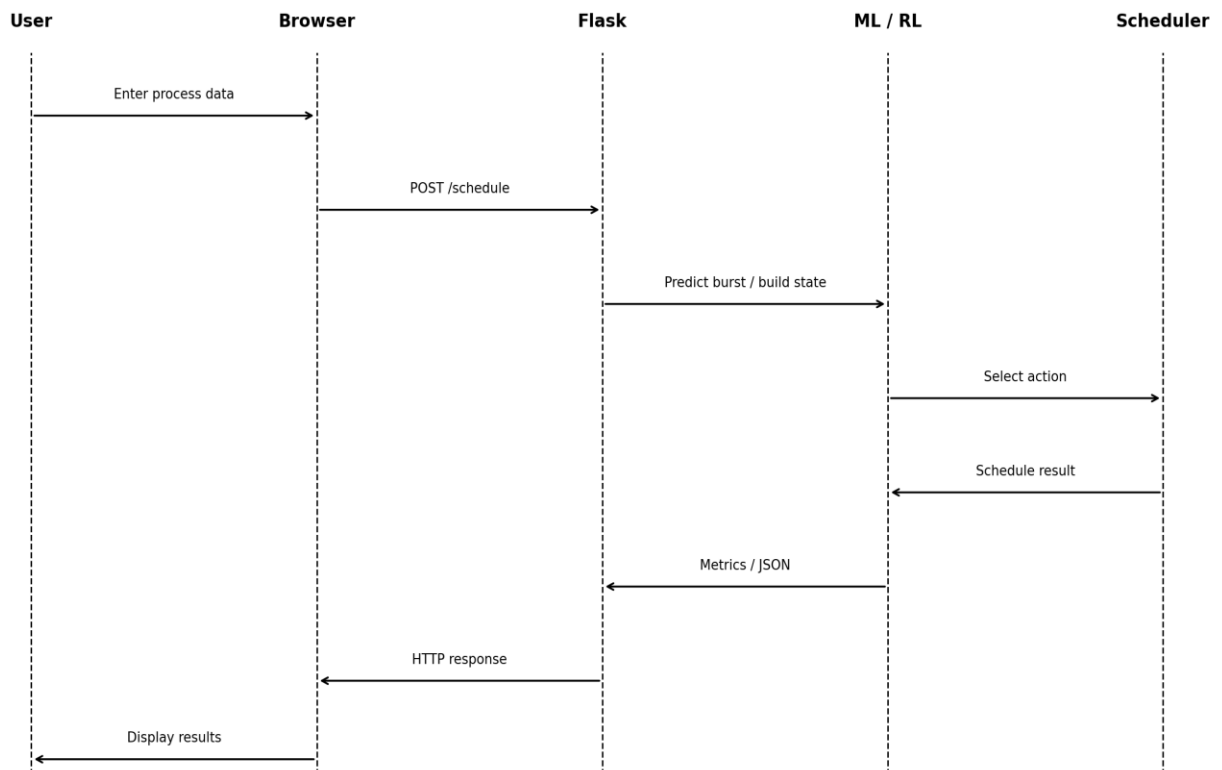


Figure 7.3 — AI Scheduling Request Sequence

7.4 Frontend Architecture

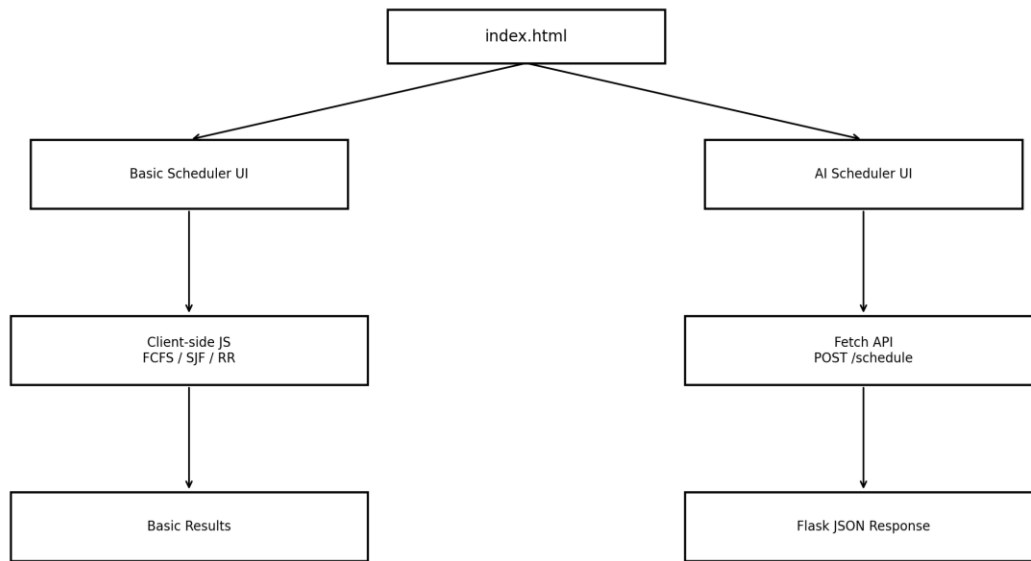


Figure 7.4 — Corrected Frontend Architecture

8. FUNCTIONAL MODULES

8.1 Traditional Scheduler Module

The Basic Scheduler interface provides FCFS, SJF, and Round Robin calculations in client-side JavaScript. The module accepts process identifiers, arrival times, burst times, and, where applicable, a time quantum. It calculates schedule information and performance metrics for display.

8.2 AI Prediction Module

The burst-time prediction module loads process data, selects the model features, applies StandardScaler, trains or loads a RandomForestRegressor, and produces a burst-time estimate. The model and scaler are serialized using Joblib.

8.3 RL Scheduling Module

The RL module defines ProcessSchedulingEnv and connects it with a PPO agent. The environment state includes predicted burst time, waiting time, queue length, and progress. The action space contains two discrete actions: continue the current process or switch to another process.

8.4 Web API Module

The Flask backend exposes the /schedule endpoint for AI scheduling requests. The frontend uses fetch to send JSON and receives the scheduling result as JSON.

9. AI/ML AND REINFORCEMENT LEARNING IMPLEMENTATION

9.1 Machine Learning Burst-Time Prediction

The ML component uses a Random Forest Regressor. The project dataset is processed into input features and a target representing CPU burst time. The implementation applies an 80/20 train-test split and evaluates predictions on the test partition using MAE, MSE, and R^2 . Therefore, a reported R^2 value should be described as a test-set metric when it comes from the evaluation code, not as a training-only score.

9.2 Feature Set

The AI interface collects arrival time and process-related measurements including I/O write bytes, voluntary context switches, CPU percentage, I/O read bytes, I/O read count, and I/O write count. These values are used as inputs to the burst-time prediction workflow.

9.3 Reinforcement Learning State

State Variable	Meaning
Predicted burst	Burst-time estimate supplied to the environment.
Waiting time	Time accumulated while the process waits.
Queue length	Number of processes currently represented in the scheduling queue.
Progress	Current execution progress of the process.

9.4 Action Space

Action	Meaning
0	Continue current process
1	Switch process

9.5 Reward Function

Event	Reward/Penalty
Process completion	+5.0
Context switch	-0.5
Excessive waiting	-0.1

9.6 PPO Configuration

Parameter	Value
learning_rate	0.0001
n_steps	1024
batch_size	32
n_epochs	8
gamma	0.99

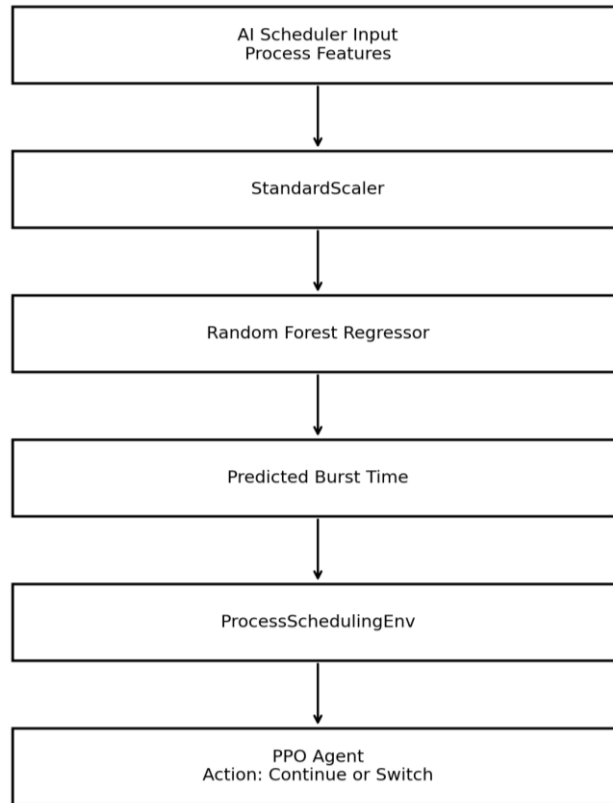


Figure 9.1 — Corrected AI/ML and Reinforcement Learning Pipeline

The AI pipeline should be interpreted as a simulation workflow. It is not an implementation of a kernel scheduler and should not be described as a guaranteed optimal scheduling policy.

10. DATA STORAGE DESIGN

The project does not use a relational database. Runtime scheduling state is handled in memory, while the machine-learning workflow uses local CSV data and serialized model/scaler artifacts. The previous heading 'Database Design' has therefore been changed to 'Data Storage Design'.

Storage	Purpose
data/process_data.csv	Training/evaluation dataset for burst-time prediction.
models/	Serialized ML predictor/scaler and RL model artifacts as used by the project.
Runtime memory	Temporary process and scheduling state during execution.

10.1 Limitation

There is no persistent user/session database for storing historical scheduling experiments. A future SQLite or PostgreSQL layer could be added if persistent experiment tracking is required.

11. API DESIGN

11.1 POST /schedule

Property	Description
Method	POST
Endpoint	/schedule
Purpose	Runs the AI scheduling workflow and returns scheduling results.
Request	JSON containing the selected algorithm and process feature objects.
Processing	Flask → burst-time prediction → RL environment/PPO → metrics.
Response	JSON containing the schedule and available performance metrics.

11.2 API Flow

The browser constructs the request from the AI Scheduler form and sends it with fetch. Flask parses the JSON payload, prepares the process data, runs the AI scheduling pipeline, calculates result information, and returns JSON to the browser. The browser then renders the AI results table and metrics.

The API section has intentionally been kept limited to the endpoint actually documented by the project rather than inventing additional endpoints.

12. USER INTERFACE

The interface uses HTML, CSS, JavaScript, and Bootstrap-based layout. It presents two primary areas: a Basic Scheduler and an AI-Powered Scheduler.

12.1 Basic Scheduler Output

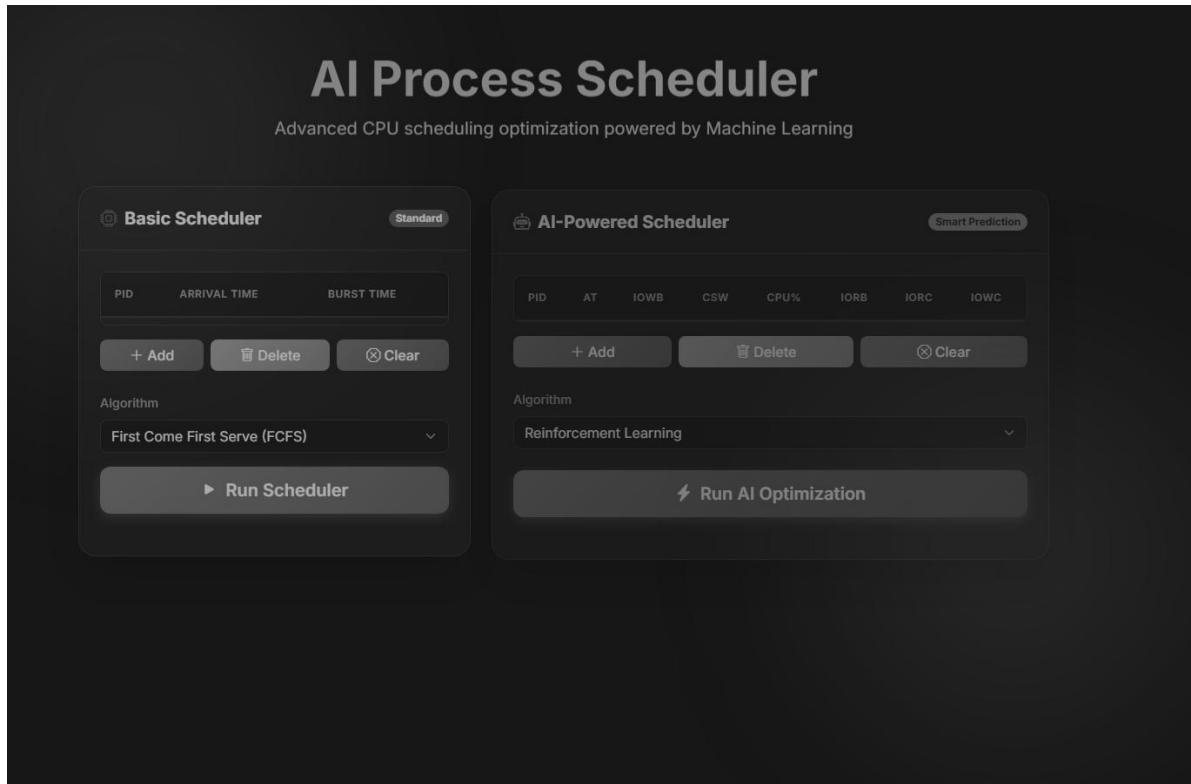


Figure 12.1 — Application Home Interface (grayscale reproduction of the supplied project screenshot)

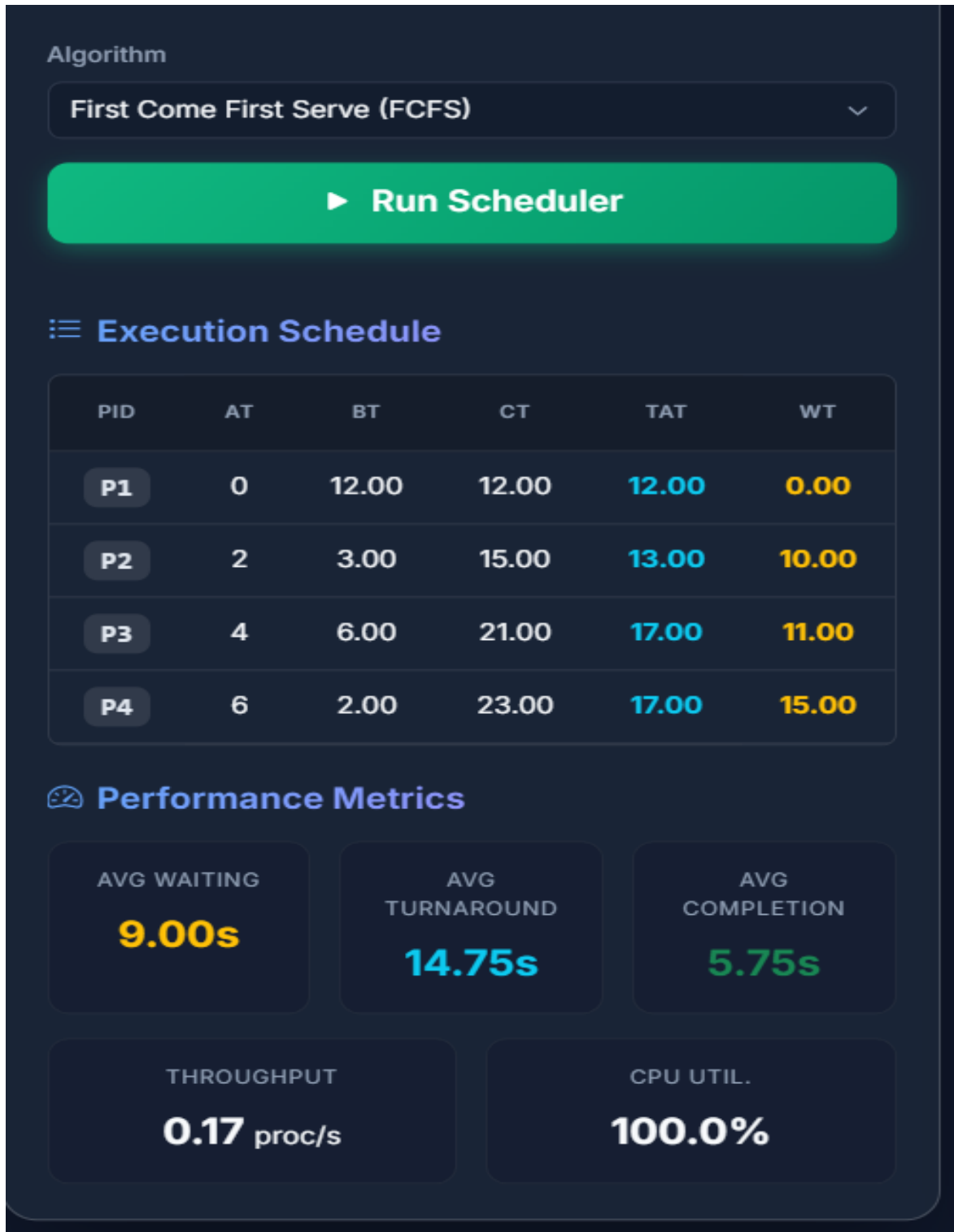


Figure 12.2 — Scheduler Execution and Performance Metrics (supplied project screenshot; grayscale)

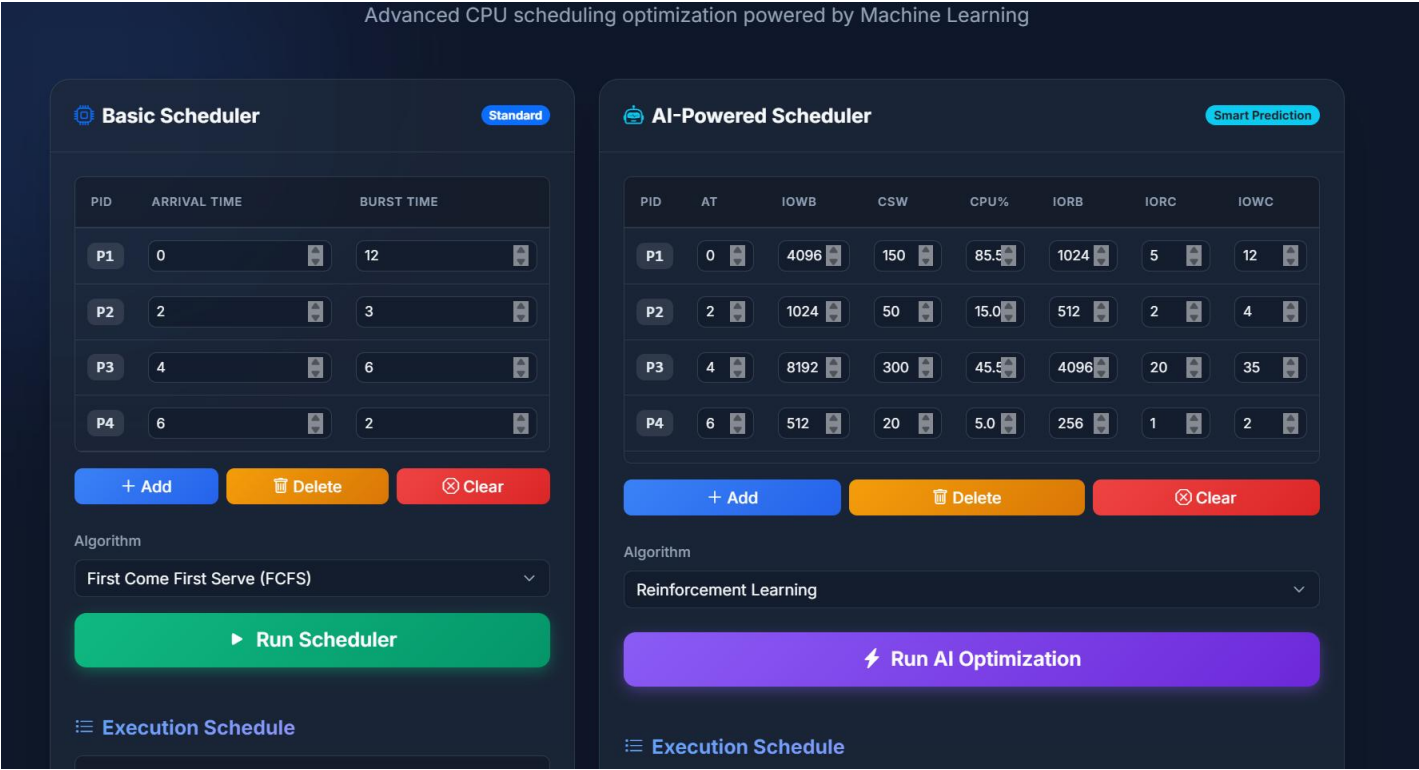


Figure 12.3



Figure 12.4

The supplied execution screenshot demonstrates the two-panel interface and shows schedule/metric output. The AI panel in the captured run displays CPU utilization as 'NaN%'. This is an observed output of the supplied screenshot and should be treated as a calculation/edge-case issue rather than as a valid measured CPU-utilization result.

12.2 UI Inputs

- Basic Scheduler: process ID, arrival time, burst time, algorithm selection, and Round Robin quantum where applicable.
- AI Scheduler: process ID, arrival time, I/O write bytes, context-switch information, CPU percentage, I/O read bytes, I/O read count, and I/O write count.
- Execution controls: add, delete, clear, run scheduler, and run AI optimization.

13. IMPLEMENTATION DETAILS

13.1 Frontend Implementation

index.html contains the interface and client-side scheduling logic. The basic scheduler functions execute in the browser. The AI scheduler uses the Fetch API to communicate with Flask.

13.2 Backend Implementation

app.py starts the Flask application, serves the web page, receives AI scheduling requests, and coordinates the ML and RL components. The backend is therefore responsible for the AI path rather than for all scheduler calculations.

13.3 Machine Learning Implementation

models/ml_burst.py handles dataset loading, feature preparation, train-test splitting, scaling, Random Forest training/evaluation, and model serialization. The evaluation logic includes MAE, MSE, and R^2 .

13.4 Reinforcement Learning Implementation

scheduler/rl_scheduler.py defines the custom scheduling environment and PPO workflow. The environment maintains process state, calculates rewards, and exposes a discrete action space for continue/switch decisions.

13.5 Important Architecture Correction

The original document stated that traditional schedulers were part of the Flask backend. The implementation instead performs the basic scheduling algorithms on the client side. The corrected architecture and descriptions in this report reflect that distinction.

14. SECURITY

14.1 Current Security Position

The application is primarily designed as a local simulation. The documentation does not identify a full production authentication/authorization system, persistent user database, or a comprehensive API security layer.

14.2 Input Validation Considerations

- Validate JSON structure before processing.
- Limit the number and size of process records accepted by the endpoint.
- Validate numeric ranges for process features.
- Handle malformed requests without exposing internal stack traces.
- Use production-grade secret management if external services are introduced.

14.3 Scope of Security Claims

Security improvements listed here are recommendations. They should not be presented as already implemented unless the corresponding code is added and verified.

15. TESTING

15.1 Current Testing

The project is primarily demonstrated through manual UI execution and the evaluation facilities in the RL/ML components. No comprehensive automated application test suite is documented in the supplied project report.

15.2 Recommended Test Matrix

Test ID	Module	Test Case	Expected Result
T01	FCFS	Enter valid processes and run FCFS	Valid execution schedule and metrics
T02	SJF	Enter valid processes and run SJF	Shortest-job ordering under the simulated assumptions
T03	Round Robin	Run with a valid quantum	Time-sliced schedule is produced
T04	AI API	Send valid JSON to /schedule	JSON scheduling result is returned
T05	ML	Run predictor evaluation	MAE, MSE and R^2 are produced
T06	RL	Run PPO environment	Agent produces valid continue/switch actions
T07	Validation	Send malformed/oversized input	Request is rejected safely

15.3 Observed UI Issue

The supplied screenshot of the AI scheduler shows CPU utilization as NaN%. This should be corrected in the implementation or explicitly recorded as a known test defect. It should not be used as evidence of successful CPU-utilization measurement.

16. DEPLOYMENT

16.1 Local Setup

1. Create and activate a Python virtual environment.
2. Install the packages specified by the project's requirements file.
3. Ensure the dataset and serialized model files are available in their expected project directories.
4. Run the Flask application using the project's app.py entry point.
5. Open the local Flask address in a modern web browser.

16.2 Example Local Execution

The supplied project documentation uses ``python app.py`` and a local Flask address. The exact dependency versions should be taken from the repository's requirements file when reproducing the project; they should not be replaced with inferred versions.

16.3 Deployment Scope

The current architecture is suitable for local demonstration and experimentation. Production deployment would require hardened configuration, input validation, monitoring, controlled model loading, and appropriate web-server deployment practices.

17. LIMITATIONS

- The project is a CPU-scheduling simulation rather than a kernel-level operating-system scheduler.
- The basic schedulers execute client-side while the AI scheduler uses the Flask backend, so the two paths are architecturally different.
- The ML model depends on the available dataset and therefore requires independent validation before claims of generalization.
- The current storage approach does not provide persistent experiment history.
- The displayed AI screenshot contains a NaN CPU-utilization result, indicating an edge case that requires correction.
- Arrival-time and workload realism should be evaluated further before treating the RL results as representative of real operating-system workloads.
- Performance comparisons should be based on repeated controlled experiments rather than a single screenshot.

18. FUTURE ENHANCEMENTS

1. Add persistent experiment storage using SQLite or PostgreSQL.
2. Improve validation for arrival times, feature ranges, and process counts.
3. Investigate and correct the CPU-utilization NaN edge case.
4. Add repeatable benchmark workloads and confidence/variance reporting.

5. Evaluate the ML model using independent validation/test traces.
6. Train and evaluate the RL scheduler with broader workload distributions and real process traces where appropriate.
7. Extend the simulator to multi-core CPU scheduling.
8. Add automated unit and integration tests.
9. Improve production deployment and security boundaries.

19. CONCLUSION

The AI-Driven Process Scheduling System demonstrates an experimental integration of conventional CPU scheduling with machine learning and reinforcement learning. The application provides a browser interface for FCFS, SJF, and Round Robin scheduling and a separate AI workflow in which a Random Forest model estimates burst time and a PPO agent interacts with a custom scheduling environment.

The revised architecture clarifies that conventional scheduling calculations are performed on the client side, while the AI workflow is handled by Flask. The revised report also distinguishes machine learning and reinforcement learning from LLM-based systems, replaces unsupported claims of optimality with implementation-focused descriptions, and records the observed CPU-utilization NaN issue in the supplied screenshot.

The project provides a useful academic platform for exploring AI-assisted scheduling. To establish stronger performance conclusions, future work should include independent model validation, repeatable benchmark workloads, corrected metric calculations, automated tests, and broader workload evaluation.

REFERENCES

- Python Flask documentation.
- scikit-learn documentation for Random Forest Regressor and Standard Scaler.
- Stable-Baselines3 documentation for PPO.
- Gym/Gym-compatible reinforcement-learning environment documentation.
- Bootstrap 5 documentation.